# Teknisk audit – Mellemrum

| Område | Fund og evidens | Konsekvens | Forslag til løsning | Prioritet | Planlagt verifikation |
|---|---|---|---|---|---|
| BRANCH 1<br>Datamodel – venues | Venue-oplysninger som navn, adresse, by og website ligger direkte på hvert event og gentages på tværs af events. | Samme data gemmes flere steder og kan blive inkonsistent, hvis oplysninger om et venue ændres. | Opret en separat venues-tabel og forbind events til den med venueId som foreign key. | Høj | Ændr oplysninger på et venue i Supabase og kontrollér, at ændringen automatisk vises på de events, der bruger venuet. |
| Datamodel – tilmeldinger | Registrations gemmer kopier af eventTitle, eventDate og eventLocation og har ikke en direkte relation til eventet. | Eventdata findes flere steder og kan derfor blive forskellige eller forældede. | Tilføj eventId til registrations og forbind den til events.id med en foreign key. Fjern derefter de gamle kopierede eventdata. | Høj | Ændr titlen på et event i Supabase og kontrollér, at den nye titel automatisk vises på /tilmeldinger. |
| Tilmeldingsflow | Formularen logger kun navn og email med console.log. Der bliver ikke oprettet en tilmelding i Supabase. | Brugeren kan udfylde formularen, men tilmeldingen bliver ikke gemt. | Lav et POST-request til Supabase REST API, som gemmer navn, email og det valgte eventId i registrations. | Høj | Foretag en tilmelding gennem appen og kontrollér i Supabase, at der bliver oprettet en ny registration med korrekt eventId. |
| Intern tilmeldingsside | /tilmeldinger viser de kopierede eventdata fra registrations frem for data fra det relaterede event. | Siden kan vise forældede eventdata, hvis et event efterfølgende bliver ændret. | Hent registrations sammen med relaterede events og venues gennem Supabase REST API med nested select. | Høj | Kontrollér Network-response i DevTools og se, at event og venue bliver hentet som relateret/nested data. |
| Feedback ved tilmelding | Formularen giver ikke feedback, mens tilmeldingen bliver sendt, eller hvis requestet lykkes eller fejler. | Ved langsom internetforbindelse kan brugeren være i tvivl om, hvorvidt der sker noget, og kan forsøge at sende formularen flere gange. | Tilføj en submitting/loading-state, deaktivér knappen under requestet og vis success- eller error-feedback bagefter. | Mellem | Simulér langsomt internet i DevTools og kontrollér, at loading/submitting-state vises, indtil Supabase svarer. Test også succes og fejl. |
| Formular / tilgængelighed | Formularfelterne mangler korrekt browservalidering, fx type="email" og required. | Brugeren kan forsøge at sende manglende eller ugyldige oplysninger. | Brug required på nødvendige felter og type="email" på emailfeltet. | Mellem | Forsøg at sende formularen uden navn/email og med en ugyldig email og kontrollér, at browseren stopper submit. |
| React-arkitektur / Supabase | Supabase URL, headers og REST-logik bliver gentaget i flere pages som HomePage, EventPage og RegistrationsPage. | Der opstår duplikeret kode, og fremtidige ændringer skal potentielt laves flere steder. | Saml Supabase REST-logikken i en fælles service/helper, så pages ikke selv skal opsætte den samme request-logik. | Mellem | Kontrollér efter refaktorering, at alle sider stadig henter data korrekt, og at den gentagne Supabase-kode er reduceret. |
| CRUD – Update | På /tilmeldinger kan status på en registration ikke ændres. Tilmeldinger kan kun vises, og ændringer skal derfor foretages manuelt i Supabase. | Den interne side kan ikke bruges til at behandle eller administrere tilmeldinger direkte. Det gør arbejdsgangen mere besværlig og afhængig af Supabase. | Tilføj mulighed for at ændre status på en registration via Supabase REST API med et PATCH-request. UI'et skal opdateres direkte efter succes og vise loading/error-feedback under ændringen. | Mellem | Ændr status på en tilmelding på /tilmeldinger og kontrollér, at ændringen vises direkte i UI og samtidig bliver gemt i registrations i Supabase. Reload siden og kontrollér, at den nye status stadig vises. |
| CRUD – Delete | På /tilmeldinger er det ikke muligt at slette en registration. En tilmelding kan derfor kun fjernes manuelt direkte i Supabase. | Den interne side mangler mulighed for at rydde op i eller fjerne ugyldige/fejlregistreringer, og administrationen er derfor ufuldstændig. | Tilføj en slet-funktion via Supabase REST API med et DELETE-request. Brugeren skal bekræfte handlingen, og registrationen skal fjernes fra UI'et efter succes. Der skal også være loading- og error-feedback. | Mellem | Slet en testtilmelding på /tilmeldinger og kontrollér, at den forsvinder fra UI'et og fra registrations i Supabase. Reload siden og kontrollér, at tilmeldingen stadig er væk. |
| UI-state – Loading | De datadrevne sider viser ikke tydelig feedback, mens data hentes fra Supabase. På langsomme forbindelser kan brugeren derfor opleve, at der ikke sker noget. | Brugeren kan blive i tvivl om, hvorvidt siden arbejder eller er gået i stå. | Tilføj en loading-state på de datadrevne sider, fx “Henter events…” og “Henter tilmeldinger…”, mens requestet er i gang. | Mellem | Simulér langsom forbindelse i Chrome DevTools og kontrollér, at loading-beskeden vises, indtil Supabase svarer. |
| UI-state – Error | Hvis et API-request fejler, håndterer siden ikke nødvendigvis fejlen tydeligt i UI'et. | Brugeren kan møde en tom eller uforståelig side uden at vide, hvad der er sket. | Tilføj en error-state med en forståelig fejlbesked, når et fetch-request fejler eller returnerer et ugyldigt response. | Høj | Blokér Supabase REST-requestet i Chrome DevTools og kontrollér, at siden stadig loader og viser en fejlbesked i stedet for at crashe. |
| UI-state – Empty | Hvis et request lykkes, men returnerer 0 resultater, er der ikke nødvendigvis en tydelig empty-state. | Brugeren kan tro, at siden er defekt, selvom der blot ikke findes noget data. | Tilføj en separat empty-state, der forklarer, at der ikke findes events eller tilmeldinger. | Mellem | Brug midlertidigt et filter i requestet, som giver 0 resultater, og kontrollér at Supabase svarer 200 OK, mens empty-state vises. |
| React-arkitektur – Footer | Footeren er ikke samlet som en genbrugelig komponent på tværs af applikationens sider. Det betyder, at samme UI og indhold kan ende med at blive gentaget flere steder. | Gentaget kode gør løsningen sværere at vedligeholde og øger risikoen for, at footeren bliver forskellig fra side til side. | Opret en selvstændig Footer-komponent og genbrug den på de relevante sider. Det følger React-princippet om component-based architecture og reducerer duplikeret JSX. | Mellem | Gennemgå alle routes og kontrollér, at footeren vises ens på alle relevante sider, at links virker, og at der ikke opstår fejl i konsollen. |

| Område | Fund og evidens | Konsekvens | Forslag til løsning | Prioritet | Planlagt verifikation |
|---|---|---|---|---|---|
| Navigation – Scroll-position | Hvis brugeren er scrollet langt ned på én side og navigerer til en ny route, kan den tidligere scroll-position blive bevaret. Den nye side kan derfor åbne et stykke nede i indholdet. | Brugeren kan gå glip af sidens overskrift og første indhold. Navigationen kan føles uforudsigelig og mindre naturlig. | Implementér en genbrugelig ScrollToTop-komponent, som registrerer ændringer i pathname via React Router og kalder window.scrollTo(0, 0) ved route-skift. | Mellem | Scroll langt ned på en side, navigér til flere forskellige routes, og kontrollér hver gang, at den nye side starter øverst. Kontrollér også, at normal navigation fortsat virker. |
| Semantisk HTML – Formular | Ved gennemgang af projektets semantiske HTML blev e-mailteksten på EventPage fundet som et generisk span-element. Teksten fungerede visuelt som label til e-mailfeltet, men var ikke semantisk forbundet med inputfeltet. Den øvrige struktur var overordnet korrekt med links til navigation, buttons til handlinger og ét naturligt main-element pr. route. | Et span fortæller ikke browseren eller hjælpemidler, at teksten beskriver et inputfelt. Det giver en mindre tydelig formularstruktur og dårligere understøttelse af bl.a. skærmlæsere. | Erstat span-elementet med et rigtigt label-element og forbind det med e-mail-inputtet gennem htmlFor og id. Bevar den eksisterende styling ved at flytte CSS-stylingen til den nye label-class. | Mellem | Kontrollér at label og input er korrekt forbundet, at designet er uændret, og at formularen fortsat fungerer. Kør npm run lint, npm run build og git diff --check. Alle tre kontroller skal bestå. |
| Søgning / filtrering | Den eksisterende event-søgning matchede titel, summary og venue-navn. Første forsøg udvidede søgningen med event.tags, men testen viste, at søgning efter fx “koncert” stadig ikke fandt de forventede events. En ny gennemgang viste, at værdier som “Koncert” og “Scene” ligger i event.category og ikke i event.tags. | Relevante events kunne mangle i søgeresultaterne, selvom de var kategoriseret med det ord, brugeren søgte efter. Det gjorde søgningen mindre intuitiv og gav ufuldstændige resultater. | Udvid søgelogikken, så den samlede søgetekst omfatter titel, summary, category, venue-navn og tags. Bevar case-insensitive søgning og sikker håndtering af manglende tags. | Høj | Test specifikt søgningerne “koncert”, “Koncert”, “scene” og “Scene” og kontrollér, at events med de pågældende kategorier vises. Kontrollér også søgning på titel, summary, venue og tags samt events uden tags. |
| Sortering / informationsarkitektur | På den interne side med tilmeldinger blev registrations vist i én fast rækkefølge. Brugeren kunne ikke sortere listen efter navn, event eller dato. Det gjorde det sværere at skabe overblik, især når antallet af tilmeldinger vokser. | Brugeren skal bruge mere tid på manuelt at finde bestemte personer, events eller datoer. Det reducerer overblikket og gør administrationen mindre effektiv. | Gør kolonneoverskrifterne Navn, Event og Dato interaktive. Et klik på overskriften sorterer listen, og gentagne klik skifter retning. Brug localeCompare med dansk locale til tekst og dato-sammenligning til eventdato. Vis en pil, der angiver den aktive sorteringsretning. | Mellem | Klik på Navn og kontrollér A-Å og Å-A. Klik på Event og kontrollér A-Å og Å-A. Klik på Dato og kontrollér nyeste/ældste. Test derefter statusændring og sletning i en sorteret liste og kontrollér, at CRUD stadig virker. Kør lint, build og git diff --check. |
| Formular / validering | Tilmeldingsformularen skal sikre, at navn og e-mail er udfyldt korrekt, før en tilmelding sendes til Supabase. Uden validering kan brugeren fx forsøge at sende et tomt felt eller en e-mailadresse uden et gyldigt e-mailformat. | Ugyldige eller mangelfulde oplysninger kan blive forsøgt sendt, hvilket giver dårligere datakvalitet og en mindre tydelig brugeroplevelse. | Brug HTML5-formularvalidering med required på obligatoriske felter og type="email" på e-mailfeltet. Browseren kan dermed stoppe formularen og give brugeren direkte feedback ved fx manglende @ i e-mailadressen. | Mellem | Forsøg at sende formularen med tomme obligatoriske felter og kontrollér, at browseren stopper submit. Indtast derefter en e-mail uden @ og kontrollér, at browserens valideringsbesked vises. Test til sidst med gyldigt navn og e-mail og kontrollér, at tilmeldingen kan gennemføres. |
| Robusthed / fejlhåndtering | Ved fejl under hentning af events og tilmeldinger vises en error-state, men brugeren har ikke en direkte mulighed for at forsøge requestet igen. For at komme videre skal brugeren derfor manuelt genindlæse hele siden. | En midlertidig netværks- eller API-fejl kan efterlade brugeren i en dead end, selv hvis forbindelsen virker igen kort efter. Det gør løsningen mindre robust og giver en dårligere recovery-oplevelse. | Tilføj en “Prøv igen”-knap i event-sidens error-state og det samme i tilmeldinger siden. Knappen genbruger den eksisterende funktion, der henter events, nulstiller fejl-state og aktiverer loading-state. Der sendes kun et nyt request, når brugeren aktivt vælger at prøve igen. | Mellem | Fremprovokér en fejl ved at blokere event-requestet i DevTools. Kontrollér at fejlbesked og “Prøv igen”-knap vises. Fjern blokeringen og klik “Prøv igen”. Kontrollér at loading-state vises, og at events derefter hentes uden fuld reload. Test også en ny fejl samt eksisterende søgning og navigation. |
| Formular / validering | Tilmeldingsformularen kunne forsøges indsendt uden navn og e-mail, og e-mailfeltet kontrollerede ikke det grundlæggende e-mailformat. | Der er risiko for mangelfulde eller ugyldigt formaterede oplysninger og en mindre robust tilmeldingsproces. | Gør navn og e-mail obligatoriske med `required`, og brug `type="email"` på e-mailfeltet. Dermed anvendes browserens indbyggede HTML5-validering før formularen sendes. | Høj | Test submit uden navn, uden e-mail og med e-mail uden @. Formularen skal stoppes. Test derefter med gyldigt navn og e-mail og kontrollér, at tilmeldingen stadig gemmes korrekt i Supabase. |
| Routing / URL-struktur | Individuelle events blev identificeret med database-ID i URL'en, fx /events/2. URL'en gav derfor ingen information om, hvilket event siden omhandlede. | URL'erne er mindre forståelige for brugeren og er tæt koblet til interne database-ID'er frem for eventets offentlige identitet. | Tilføj et permanent og unikt slug-felt til events i Supabase. Brug /events/:slug i React Router, link eventkort med event.slug, og hent EventPage via slug. Behold event.id til database-relationer som registrations.eventId. | Mellem | Klik på flere eventkort og kontrollér slug-URL'erne. Genindlæs direkte på en slug-URL. Test en slug, der ikke findes. Kontrollér at korrekt event og venue vises, og at tilmeldingsformularen stadig opretter registration med korrekt eventId. Kør lint, build og git diff --check. |

| | | | | | |
|---|---|---|---|---|---|
| UX / navigation | På eventkortene skulle brugeren klikke specifikt på "Læs mere" for at åbne eventet, selvom hele kortet visuelt repræsenterer det samme event. | Det relativt lille klikområde gør navigationen mindre effektiv og kræver større præcision fra brugeren. | Gør hele eventkortet til et semantisk korrekt React Router Link til eventets slug. Bevar "Læs mere" visuelt, men undgå nested links. Sørg samtidig for synlig fokus-state og tastaturnavigation. | Mellem | Klik på billede, titel, tomt område og 'Læs mere' på flere kort. Kontrollér korrekt slug-URL. Test derefter med Tab + Enter, mobil/desktop samt eksisterende søgning. Kør lint, build og git diff --check. |
| CSS-arkitektur / vedligeholdelse | Projektet anvender én global styles.css på 999 linjer. Footerens base-, hover- og responsive regler ligger her, selv om de kun bruges af Footer.jsx. Auditen viste derfor et tydeligt, afgrænset område, der kan flyttes uden at påvirke andre komponenter. | Global styling gør ejerskabet mindre tydeligt og øger risikoen for class name-konflikter og utilsigtede påvirkninger, efterhånden som projektet vokser. Det kræver også, at man leder flere steder i en stor stylesheet for at ændre én komponent. | Flyt kun Footerens egne regler til src/components/Footer.module.css og opdater Footer.jsx til at bruge det importerede styles-objekt. Bevar globale designvariabler, reset og fælles styles i styles.css. Ingen andre komponenter konverteres i denne branch. | Mellem | Sammenlign Footer før/efter på alle relevante sider. Test links, hover, desktop og mobil omkring 620 px. Kontrollér at andre komponenter er visuelt uændrede. Kør npm run lint, npm run build og git diff --check. |
| CSS-arkitektur / Navbar | Navbarens komponent-specifikke styling ligger i den globale styles.css sammen med styling til resten af applikationen. Auditen identificerede bl.a. .site-nav, .brand og .nav-links som regler, der kun tilhører Navbar. | Stylingen har global rækkevidde, hvilket gør ejerskab mindre tydeligt og øger risikoen for class name-konflikter, når projektet udvikles. | Flyt kun Navbarens base-, hover-, active- og responsive styling til Navbar.module.css. Opdater Navbar.jsx til at bruge det importerede styles-objekt. Bevar globale designvariabler og fælles styles i styles.css. | Mellem | Sammenlign Navbar visuelt før/efter. Test alle links, aktiv route, hover, Tab + Enter, direkte reload på routes samt desktop og mobil. Kontrollér at andre komponenter er upåvirkede. Kør lint, build og git diff --check. |
| Lighthouse performance | Tre mobile baseline-tests gav performance 78 / 65 / 82. Efter billedoptimeringen gav tre nye tests 90 / 90 / 92. | Baseline viste både forbedringspotentiale og stor variation. Eftermålingen viser en højere og mere stabil performance-score. | Optimér den dokumenterede flaskehals frem for at jagte en bestemt Lighthouse-score. | Høj | Udført: samme deployede route blev testet tre gange før og tre gange efter. Median performance steg fra 78 til 90. |
| Deployment / SPA-routing | Ved direkte adgang eller reload af en event-URL som /events/lyd-mellem-husene viste Chrome Network tidligere et rødt document-request med status 404. Problemet skyldtes, at GitHub Pages forsøgte at finde den dynamiske React-route som en fysisk fil. | Selvom React kunne vise siden efterfølgende, var deep linking og reload ikke håndteret korrekt i deployment-opsætningen. | Der er implementeret en GitHub Pages SPA-fallback med 404.html og route-gendannelse i index.html. Løsningen tager højde for projektets base path og bevarer BrowserRouter samt de læsbare slug-URL'er uden #. | Høj | Efter deployment blev den samme event-URL åbnet direkte med Network aktivt. Document-requestet returnerer nu 200, og de centrale requests til Supabase, JS, CSS og eventbillede returnerer også 200. Den tidligere røde 404 ses ikke længere. |
| Performance / EventPage | Tre Lighthouse Mobile-tests gav performance 64 / 65 / 65, LCP 4,1 / 3,9 / 3,9 s og CLS 0,671 i alle tre tests. Analyse viste, at footeren flyttede sig kraftigt, når loading-state blev erstattet af eventdata, og at eventbilledet var LCP-elementet. | Det store layout shift gjorde siden visuelt ustabil under indlæsning, mens LCP viste, at det primære eventbillede blev vist senere end ønsket. | EventPage fik en stabil minimumshøjde under loading. Eventbilledet fik responsive srcSet/sizes, width/height og fetchPriority="high", mens eager loading, crop og design blev bevaret. | Høj – færdig | Efter deployment blev samme route testet tre gange igen: performance 94 / 93 / 93, LCP 3,0 / 3,1 / 3,1 s og CLS 0 / 0 / 0. FCP og TBT forblev gode. Forbedringen er dermed verificeret med Lighthouse. |
| Accessibility / Keyboard navigation | Audit viste, at de fleste interaktive elementer allerede brugte native links, buttons, inputs og selects. Der manglede dog et skip-link til hovedindholdet, og fokusmarkeringen på Navbar, Footer og tilmeldingsknappen var mindre tydelig. | Keyboard-brugere skulle navigere gennem den samme Navbar på hver side, og det kunne enkelte steder være svært at se, hvilket element der havde fokus. | Tilføj et synligt-ved-fokus “Spring til hovedindhold”-link, der peger på <main id="main-content"> med tabIndex={-1}. Tilføj samtidig tydelige :focus-visible-states på relevante links og knapper. | Mellem | Test uden mus: Reload → Tab viser skip-link → Enter flytter fokus til main. Test derefter Tab/Shift+Tab gennem Navbar, eventkort, formular og Footer samt Enter/Space på links og knapper. Lint, build og git diff --check skal bestå. |
| Navigation / UX | Mellemrum-logoet fungerer som navigation til forsiden, men når brugeren allerede befinder sig på forsiden og er scrollet ned, giver et klik på logoet ikke nødvendigvis en tydelig navigationseffekt. | Brugeren kan forvente, at logoet fører tilbage til starten af forsiden. Hvis siden allerede er aktiv, kan handlingen ellers opleves som om, der ikke sker noget. | Bevar logoet som link til forsiden, men sørg for, at et klik også scroller til toppen, når brugeren allerede er på forsiden. Dermed fungerer logoet både som Home-navigation og som en genvej tilbage til sidens start. | Lav–mellem | Scroll ned på forsiden og klik på Mellemrum-logoet. Kontrollér, at siden går til toppen. Test også logoet fra andre routes og kontrollér, at det fortsat navigerer korrekt til forsiden. Test med keyboard/Enter, hvis logoet er et link. |